Real-Time Relief Mapping on Arbitrary Polygonal Surfaces - IG3D Course Sorbonne University - 2025

Ko Tchong Sem ALPOU [21411880]

April 11, 2025

This project was made This project was carried out as part of the IG3D project in the Master's program in Image at Sorbonne University.

Introduction

This project is based on the paper *Real-Time Relief Mapping on Arbitrary Polygonal Surfaces* by Policarpo et al. The technique presented addresses several limitations found in traditional methods such as bump mapping, parallax mapping, and displacement mapping. These earlier techniques often suffer from visual artifacts including incorrect self-occlusion, lack of interpenetration, distorted texture projection, and inaccurate shadowing.

Relief mapping, as introduced in the paper, allows detailed textures with depth to be rendered on arbitrary polygonal surfaces while supporting realistic per-pixel lighting, self-occlusion, and perspective-correct shading. The method operates in tangent space, which allows normals stored in texture space to be correctly interpreted regardless of the surface orientation. This overcomes a key limitation of conventional normal mapping, where mismatches between surface normals and texture normals lead to unrealistic lighting results.

By leveraging tangent space, the technique supports relief textures applied to any surface orientation — vertical, horizontal, curved — without lighting artifacts. At its core, the algorithm simulates the intersection of view rays with a height field defined by a relief map, and computes adjusted UV coordinates accordingly. The technique is implemented entirely in the fragment shader, taking advantage of programmable graphics pipelines to deliver real-time performance on modern GPUs.

Data

The project uses the following input data:

- **OBJ files**: These contain the geometric description of the 3D model, including vertex positions (x, y, z), texture coordinates (u, v), and vertex normals. Texture coordinates are normalized between 0 and 1.
- **Diffuse texture**: A 2D image that provides the base color of the surface. It is sampled during the lighting computation to determine the fragment color.
- **Relief texture**: A grayscale image where pixel values are normalized between 0 and 1, representing the height (depth) of the corresponding point in the diffuse texture. This texture is used to simulate the relief effect by displacing texture coordinates during rendering.

Methods

Tangent Space

To enable per-pixel lighting and relief mapping, we implemented calculations in tangent space. For each vertex, a local coordinate system (TBN matrix: Tangent, Bitangent, Normal) was constructed.

The tangent was computed using vertex positions and texture coordinates per triangle, while the bitangent was derived via a cross product with the normal. This TBN basis allows vectors such as view direction and light direction to be transformed into the same space as the normal and height maps, which are defined in texture space. This method is essential for correctly simulating surface detail and lighting interactions at the fragment level.

0.1 Linear and Binary Search

To accurately simulate surface depth in relief mapping, we implemented a two-phase ray-heightfield intersection method. First, a linear search is performed along the view vector projected in tangent space, starting from the texture coordinate and progressing towards the direction of view. This step identifies the point where the virtual ray enters the surface, based on a height map. Once an intersection is detected, a binary search is performed within a narrow interval to refine the depth value with sub-texel precision. This combination ensures both robustness (via linear search) and efficiency (via binary search), allowing for realistic depth perception and accurate texture displacement without excessive computation.

Results

Preparation

The implementation was done using PyOpenGL, a Python wrapper for the OpenGL API. Before developing the fragment shader — the core of the relief mapping technique — we first ensured proper setup of all relevant inputs, such as texture coordinates, tangents, and normals.

Vertex Shader

The vertex shader receives each vertex's position in object space, its associated UV coordinate, the tangent vector, and the normal vector. These are used to construct the tangent space at each vertex. This space is defined by three orthonormal vectors: the tangent $\mathbf{T}$, aligned with the texture's U direction; the bitangent $\mathbf{B}$, computed as $\mathbf{B} = \mathbf{N} \times \mathbf{T}$, aligned with the V direction; and the normal $\mathbf{N}$. These form the TBN matrix, which transforms vectors from world space to tangent space. Using this matrix, we transform the vector from the camera to the fragment (view vector), and the vector from the light source to the fragment (light vector), into tangent space. This is critical because the height map and normal map are defined in this local tangent space.

Fragment Shader

The goal in the fragment shader is to compute the true texture coordinate that should be used for sampling, based on the simulated displacement caused by the height map. The incoming texture coordinate only corresponds to the flat surface; relief mapping displaces this coordinate by simulating how a ray would intersect the virtual relief geometry.

Working in tangent space, we cast a ray from the eye (camera) position in the direction of the view vector. The ray starts at the surface UV coordinate (point A) and follows a direction $\vec{V}$, projected onto the plane of the texture (UV space). We perform a **linear search** by stepping from A toward a target point $B = A + \vec{V}_{xy}$, stopping at the first position where the virtual ray penetrates the height map. We define this intersection heuristically as the point where the traveled depth t satisfies $t > h(\text{UV})$, where h is the value read from the height map.

Once the ray has entered the surface, we perform a **binary search** within the interval $[t_{\text{prev}}, t_{\text{curr}}]$ to more accurately approximate the intersection point. This significantly increases precision while keeping computation efficient.

After finding the adjusted texture coordinate UV$'$, we sample the diffuse color and the perturbed normal from the relief map. The lighting is then computed using a Blinn-Phong model: the diffuse term is proportional to $\max(0, \mathbf{N} \cdot \mathbf{L})$, where $\mathbf{N}$ is the perturbed normal and $\mathbf{L}$ the light vector. The specular term uses the half-vector $\mathbf{H} = \frac{\mathbf{L}+\mathbf{V}}{\|\mathbf{L}+\mathbf{V}\|}$, and is proportional to $\max(0, \mathbf{N} \cdot \mathbf{H})^{\alpha}$, where α controls shininess (we used $\alpha = 64$).

To maintain correct depth testing and allow proper occlusion with other scene geometry, we also compute and assign a corrected fragment depth based on the displaced surface point. This is essential for integrating relief-mapped surfaces with the rest of the 3D scene realistically.

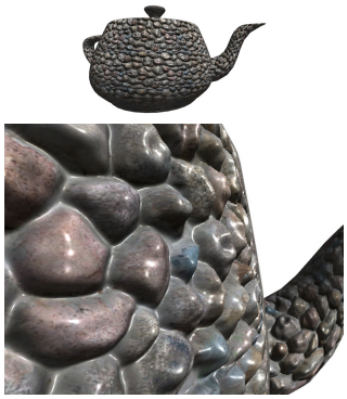

Figure 1: Teapot from the paper

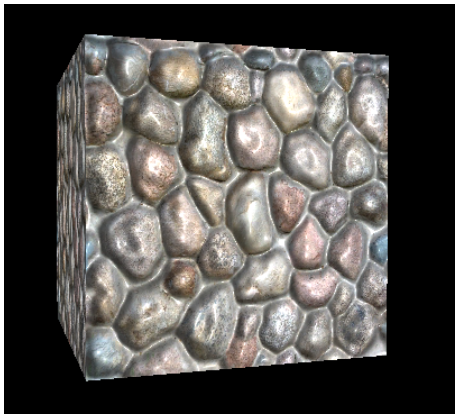

Figure 2: cube relief from our implementation

Limitations

While the core concepts from the paper *Real-Time Relief Mapping on Arbitrary Polygonal Surfaces* were successfully implemented, several limitations and challenges were encountered during development.

First, the technique as described in the paper assumes a high degree of geometric and texture coherence, particularly in tangent space construction. This requires careful preprocessing of mesh data to ensure accurate tangents and consistent UV mapping. In practice, we found that OBJ files with non-triangulated faces or inconsistent normals led to visual artifacts or incorrect shading.

Another limitation is performance-related. Relief mapping is computationally intensive, especially due to the linear and binary search loops per fragment. Although we followed the paper's guidance to balance quality and speed (e.g., 60 steps for linear search, 16 for binary), real-time performance could still be impacted on lower-end GPUs or at high resolutions.

Conclusions and Future Work

This project implemented the core relief mapping technique in a fragment shader, following the method proposed by Policarpo et al. The approach, based on projecting view rays in tangent space and performing linear and binary searches against a height map, successfully produced realistic surface depth and per-pixel lighting without modifying geometry. While the method proved effective for static surfaces and directional lighting, features such as self-shadowing and dynamic light interaction were left for future work due to time constraints. Potential extensions include implementing shadow-aware relief mapping, optimizing performance with cone step mapping, and integrating the technique into animated or interactive environments.

References

- LearnOpenGL. *Advanced Lighting: Normal Mapping*. Retrieved from: https://learnopengl.com/Advanced-Lighting/Normal-Mapping
- 3D Graphics Programming. *Relief Mapping (Advanced Parallax Mapping)*. YouTube, 2022. Retrieved from: https://www.youtube.com/watch?v=cM7RjEtZGHw